```
export default async function handler(req,
res) {
  // CORS Headers
  res.setHeader('Access-Control-Allow-
Credentials', 'true');
  res.setHeader('Access-Control-Allow-
Origin', '*');
  res.setHeader('Access-Control-Allow-
Methods',
'GET,OPTIONS,PATCH,DELETE,POST,PUT');
  res.setHeader('Access-Control-Allow-
Headers', 'X-CSRF-Token, X-Requested-With,
Accept, Accept-Version, Content-Length,
Content-MD5, Content-Type, Date, X-Api-
Version');

  if (req.method === 'OPTIONS') {
    res.status(200).end();
    return;
  }

  const GITHUB_TOKEN =
process.env.GITHUB_TOKEN;
  if (!GITHUB_TOKEN) {
    return res.status(500).json({ error:
'GITHUB_TOKEN not configured' });
  }

  const { action, owner, repo, branch,
file_path, content, message } = req.body;
```

```
  try {
    // List Repos
    if (action === 'list_repos') {
      const response = await
fetch(`https://api.github.com/users/${owner
}/repos`, {
        headers: {
          Authorization: `token
${GITHUB_TOKEN}`,
          Accept:
'application/vnd.github.v3+json'
        }
      });

      if (!response.ok) throw new
Error('Failed to fetch repos');

      const data = await response.json();
      const repos = data.map(r => ({
        name: r.name,
        description: r.description,
        url: r.html_url,
        stars: r.stargazers_count
      }));

      return res.status(200).json({
success: true, repos });
    }

    // List Branches
    if (action === 'list_branches') {
      const response = await fetch(
```

```
`https://api.github.com/repos/${owner}/${re
po}/branches`,
        {
          headers: {
            Authorization: `token
${GITHUB_TOKEN}`,
            Accept:
'application/vnd.github.v3+json'
          }
        }
      );

      if (!response.ok) throw new
Error('Failed to fetch branches');

      const data = await response.json();
      const branches = data.map(b => ({
        name: b.name,
        protected: b.protected
      }));

      return res.status(200).json({
success: true, branches });
    }

    // Push File
    if (action === 'push_file') {
      if (!owner || !repo || !branch ||
!file_path || !content || !message) {
        return res.status(400).json({
          success: false,
          error: 'Required: owner, repo,
branch, file_path, content, message'
        });
```

```
      }

      const base64Content =
Buffer.from(content).toString('base64');

      // Check if file exists
      let sha;
      try {
        const checkRes = await fetch(

`https://api.github.com/repos/${owner}/${re
po}/contents/${file_path}?ref=${branch}`,
          {
            headers: {
              Authorization: `token
${GITHUB_TOKEN}`,
              Accept:
'application/vnd.github.v3+json'
            }
          }
        );
        if (checkRes.ok) {
          const fileData = await
checkRes.json();
          sha = fileData.sha;
        }
      } catch (e) {
        // File doesn't exist, that's ok
      }

      // Push file
      const pushRes = await fetch(

`https://api.github.com/repos/${owner}/${re
```

```
po}/contents/${file_path}`,
        {
          method: 'PUT',
          headers: {
            Authorization: `token
${GITHUB_TOKEN}`,
            Accept:
'application/vnd.github.v3+json'
          },
          body: JSON.stringify({
            message,
            content: base64Content,
            branch,
            ...(sha && { sha })
          })
        }
      );

      if (!pushRes.ok) {
        const error = await pushRes.json();
        throw new Error(`GitHub API:
${error.message}`);
      }

      const result = await pushRes.json();
      return res.status(200).json({
        success: true,
        file: result.content.name,
        commit: result.commit.sha,
        url: result.content.html_url
      });
    }

    return res.status(400).json({ error:
```

```
'Unknown action' });
  } catch (err) {
    res.status(500).json({ success: false,
error: err.message });
  }
}
```